

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO-COMPUTADORA



Proyecto Final
Escenario Deportivo 3D Interactivo: Tenis

Integrantes del equipo

Aranda Marrón Fernando

Gutiérrez Nolasco Emiliano

Chacón Jaral Hugo Emmanuel

Grupo de Laboratorio: 09

Grupo de Teoría: 05

Semestre: 2027-1

Calificación: _____

INTRODUCCIÓN

Este documento presenta el desarrollo del Proyecto Final de la asignatura Computación Gráfica e Interacción Humano-Computadora: un escenario deportivo tridimensional e interactivo dedicado al tenis. El programa está escrito en C++ con OpenGL 3.3 y reúne las técnicas estudiadas durante las prácticas del curso: carga de modelos, cámaras, modelado jerárquico, texturizado, iluminación, transparencias, animación básica, máquinas de estados y animación por keyframes.

El escenario recrea un estadio de tenis dividido en tres zonas: el terreno de juego con la red, los jugadores y la pelota; las bancas de los jugadores; y el área de espectadores, donde se ubican las gradas, la silla del juez de silla, el marcador y las torres de iluminación. El usuario puede recorrer el estadio con una cámara libre, seguir a un jugador desde atrás o desde sus propios ojos, ver el partido como una transmisión de televisión, encender los reflectores, cambiar entre día y noche, ejecutar un saque y llevar el marcador de un partido.

El trabajo se organizó con la metodología Scrum, definida en el acta del proyecto, y se dividió en tres fases a lo largo de ocho semanas. En las siguientes secciones se explican la metodología y los roles, la planeación, el diseño y la solución técnica, las pruebas realizadas, los resultados y el manual de uso, y se incluyen la liga de descarga del código fuente y la del video demostrativo.

OBJETIVOS

Objetivo general

Desarrollar un escenario deportivo 3D interactivo de tenis en OpenGL 3.3, aplicando los conocimientos de computación gráfica adquiridos en el curso y siguiendo una metodología de desarrollo ágil, de modo que el usuario pueda navegar, observar y modificar el entorno de forma sencilla.

Objetivos específicos

- Modelar y organizar al menos tres subzonas del estadio con jugadores, juez de silla, público y objetos de utilería.
- Construir a los jugadores mediante modelado jerárquico para poder animar brazos, piernas, cabeza y raqueta.
- Aplicar texturas, iluminación direccional, puntual y de reflector, con transición entre día y noche.
- Implementar cuatro cámaras: espectador, orbital en tercera persona, transmisión de televisión y primera persona del jugador.

- Programar animaciones de traslación, rotación y escala, y al menos una animación por keyframes, la del saque.
- Permitir la navegación libre y las interacciones sencillas con el entorno mediante teclado y ratón.
- Documentar la metodología, el diseño, la implementación y las pruebas, y entregar el código fuente y un video demostrativo.

METODOLOGÍA DE DESARROLLO

Selección y justificación de Scrum

Scrum es un marco de trabajo ágil que organiza el desarrollo en ciclos cortos llamados sprints, al final de los cuales se obtiene un incremento funcional del producto. Según Schwaber y Sutherland, 2020, el marco se apoya en la transparencia, la inspección y la adaptación, tres ideas que encajan con un proyecto gráfico en el que el resultado se evalúa mirando la pantalla y se ajusta de manera continua.

El equipo eligió Scrum porque el proyecto está formado por módulos que pueden entregarse y verificarse por separado, como la cancha, los jugadores, la iluminación y las cámaras. Cada módulo se integra al programa principal y se comprueba visualmente al terminar el sprint, lo que reduce el riesgo de descubrir errores de integración al final del semestre. Además, el equipo es pequeño, de tres personas, y el acta definió fases cortas con objetivos verificables.

Comparación breve con otras metodologías

Metodología	Enfoque	Ventajas	Limitaciones para este proyecto
Scrum	Iterativo e incremental, con sprints y revisiones periódicas.	Entregas frecuentes, ajuste de prioridades y buena comunicación en equipos pequeños.	Requiere disciplina en las reuniones y en la actualización del backlog.
Kanban	Flujo continuo con tablero de tareas y límite de trabajo en curso.	Muy flexible y fácil de adoptar.	No define ciclos ni fechas de revisión, y el proyecto exige fases con entregables.
Cascada	Etapas secuenciales: requisitos, diseño, implementación y pruebas.	Documentación ordenada y alcance cerrado.	Los errores de integración se descubren tarde y no admite cambios de rumbo.
Programación extrema	Ciclos muy cortos, programación en parejas y pruebas continuas.	Alta calidad técnica y retroalimentación rápida.	Exige prácticas, como la programación en parejas, difíciles de sostener con horarios distintos.

Tabla 1. Comparación de Scrum con otras metodologías de desarrollo.

Roles del equipo

Siguiendo el acta del proyecto, cada integrante se hizo responsable de un conjunto de módulos. El resto del trabajo, como la integración, las pruebas y la documentación, se repartió en las reuniones de cada sprint.

Integrante	Responsabilidades
Aranda Marrón Fernando	Modelado del escenario y de las zonas, y diseño de la interacción con el entorno.
Gutiérrez Nolasco Emiliano	Modelado jerárquico de los jugadores, texturizado e iluminación.
Chacón Jaral Hugo Emmanuel	Cámaras y animación por keyframes.

Tabla 2. Roles y responsabilidades por integrante.

Planeación por fases

El acta definió un calendario de ocho semanas dividido en tres fases, cada una tratada como un sprint con su propio objetivo y su incremento verificable.

Fase	Semanas	Objetivo	Incremento verificable
Fase 1	1 y 2	Planeación y diseño. Carga de modelos, terreno de juego y cámara libre.	Cancha, red y cámara del espectador funcionando.
Fase 2	3 a 6	Jugadores jerárquicos, texturas, iluminación y zonas de bancas y espectadores.	Estadio completo con luces, gradas y jugadores articulados.
Fase 3	7 y 8	Animación por keyframes, interacciones, marcador, pruebas, documentación y video.	Programa final, documentación y video demostrativo.

Tabla 3. Fases del proyecto con Scrum.

En cada sprint se aplicaron los eventos de Scrum de forma ligera: una planeación inicial para elegir las tareas del backlog, reuniones breves para detectar bloqueos, una revisión en la que se ejecutaba el programa para validar el incremento y una retrospectiva con los ajustes para el siguiente sprint.

ETAPA 1: PLANEACIÓN

Requerimientos y cumplimiento

La tabla siguiente relaciona cada requerimiento solicitado en la especificación del proyecto con la solución implementada.

Requerimiento	Solución en el proyecto
OpenGL 3.3 o superior	Contexto core 3.3 creado con GLFW y funciones cargadas con GLEW.

Requerimiento	Solución en el proyecto
Al menos tres subzonas	Terreno de juego, bancas de los jugadores y área de espectadores con silla del juez, marcador y torres de luz.
Jugadores, árbitro y utilería	Dos jugadores, un juez de silla, unos 430 espectadores, bancas, hieleras, sombrillas, anuncios, marcador y pelota.
Modelado jerárquico	Cada jugador se compone de torso, cabeza, brazos, antebrazos, muslos, piernas y raqueta, anidados con matrices.
Carga de modelos	Clases Model y Mesh con Assimp. Se cargan 52 modelos en formato obj.
Proyección en perspectiva	glm::perspective en todas las cámaras, con un campo de visión de 38 grados en la cámara de televisión.
Texturas	42 imágenes png con SOIL2, entre ellas la cancha con sus líneas, el pasto, el concreto y las texturas de los jugadores.
Iluminación	Luz direccional que representa el sol, cuatro luces puntuales en las torres, un reflector central y transición entre día y noche.
Al menos dos cámaras	Cuatro cámaras: espectador, orbital, transmisión de televisión y primera persona del jugador.
Animaciones de traslación, rotación y escala	Caminata de los jugadores, giro de la cabeza del juez, rebote del público, escala del marcador al cambiar un punto.
Al menos una animación por keyframes	Saque del jugador con 10 cuadros clave e interpolación lineal.
Navegación libre	Cámara del espectador con teclado y ratón, y jugador controlable dentro de la cancha.
Interacciones sencillas	Encender los reflectores, cambiar a noche, ejecutar el saque y anotar puntos.

Tabla 4. Requerimientos del proyecto y solución implementada.

Alcance

Dentro del alcance quedó todo lo descrito en la tabla anterior. Siguiendo el acta, quedaron fuera la física realista, el modo multijugador, la inteligencia artificial de los jugadores, el fotorrealismo y las reglas completas del tenis, de las cuales solo se conserva un marcador sencillo de games y puntos.

Riesgos identificados

Riesgo	Mitigación
Complejidad del modelado jerárquico y de la sincronización de la pelota con la mano y la raqueta.	Calcular la posición de la pelota con las mismas matrices del jugador y probar cada pose por separado.
Falta de modelos adecuados para el tenis.	Generar los modelos con un programa propio, de modo que todos tengan el mismo estilo y las mismas unidades.
Bajo rendimiento por la cantidad de espectadores.	Usar modelos de pocos polígonos, un modelo compartido por muchos espectadores y una caché de texturas.
Errores de integración entre módulos.	Integrar al programa principal al final de cada sprint y probarlo.

Tabla 5. Riesgos y medidas de mitigación.

ETAPA 2: DISEÑO

Arquitectura del programa

El programa sigue la estructura de las prácticas del curso. Primero se crea la ventana y se inicializan GLEW, los shaders y los modelos. Después entra al ciclo principal, que en cada cuadro calcula el tiempo transcurrido, procesa los eventos, mueve la cámara o al jugador, actualiza las animaciones, elige la matriz de vista según la cámara activa, envía las luces al shader y dibuja las tres zonas.

Módulo	Función
main.cpp	Inicialización, ciclo principal, cámaras, iluminación, animaciones, interacción y recorrido automático.
Model.h y Mesh.h	Carga de archivos obj con Assimp, manejo de materiales y caché de texturas con SOIL2.
Shader.h y Camera.h	Compilación de shaders y cámara libre, tomadas de las prácticas.
Shader/lighting.vs y .frag	Iluminación de Phong con luz direccional, cuatro luces puntuales y un reflector, y transparencia por canal alfa.
Shader/lamp.vs y .frag	Dibujo de las lámparas de los reflectores con un color uniforme.
tools/generar_modelos.py	Generador de los modelos obj, sus materiales mtl y sus texturas png.

Tabla 6. Módulos del proyecto.

Distribución del escenario

El escenario se diseñó a escala real, con una unidad igual a un metro. La cancha mide 18.29 por 36.57 metros incluyendo la zona de recorrido, la red está en el centro, en z igual a cero, y los jugadores se colocan en las líneas de fondo. Las gradas se sitúan a los lados de la cancha, las bancas junto a la red, el marcador al fondo y las cuatro torres de luz en las esquinas del estadio.

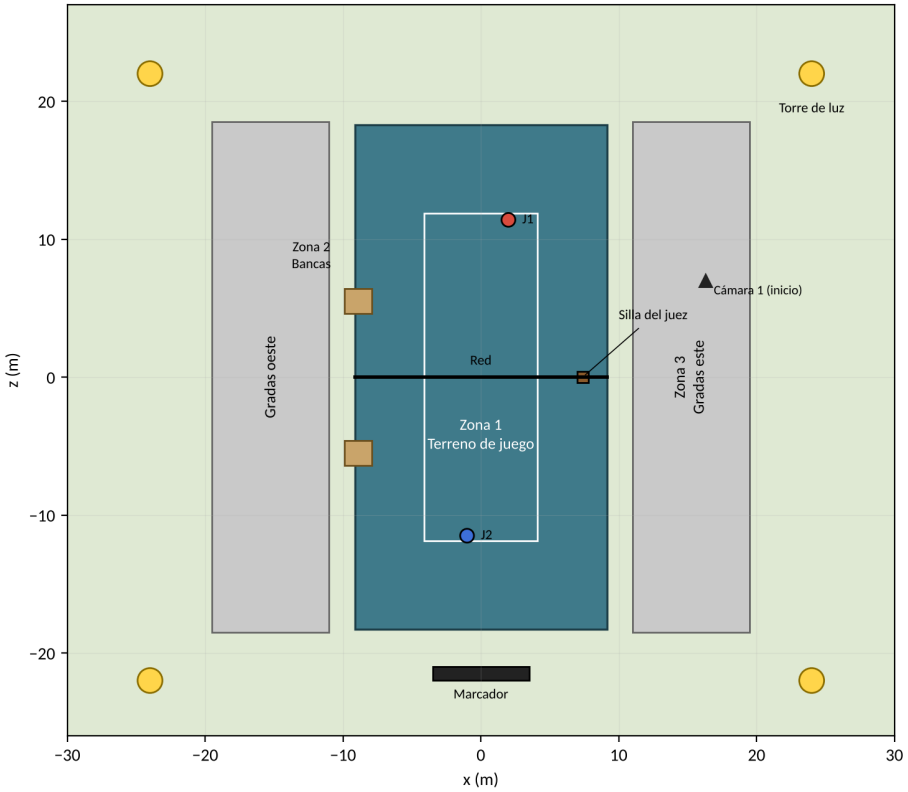


Figura 1. Planta del escenario con las tres zonas, los jugadores y las torres de luz.

Diseño de los modelos

Para que el escenario tuviera un estilo uniforme y unidades coherentes, los modelos se generaron con el programa tools/generar_modelos.py, que escribe los archivos obj, mtl y las texturas png. En total se generaron 52 modelos y 42 texturas, agrupados por zona.

Zona	Modelos
Zona 1: terreno de juego	Suelo, Cancha con sus líneas, Red con transparencia, PosteRed, tres anuncios perimetrales, Pelota y las siete partes de cada jugador, JugA y JugB.
Zona 2: bancas	Banca, Hielera y Sombrilla.
Zona 3: espectadores	Gradas, SillaJuez, Juez_Torso, Juez_Cabeza, seis modelos de espectador, Marcador, dígitos de puntos y de games, y TorreLuz.

Tabla 7. Modelos del escenario por zona.

Los anuncios y el marcador usan marcas ficticias, como ACE SPORT, FI UNAM y SMASH, para no reproducir logotipos reales.

Diseño jerárquico del jugador

Cada jugador es un árbol de transformaciones. La base fija su posición y su rumbo en la cancha. De ella cuelgan el torso, que puede girar y inclinarse, y las piernas, que dependen de la cadera. Sobre el torso se montan la cabeza y los brazos, y el brazo derecho termina en la raqueta, de modo que al rotar el hombro se mueven el antebrazo y la raqueta en conjunto.

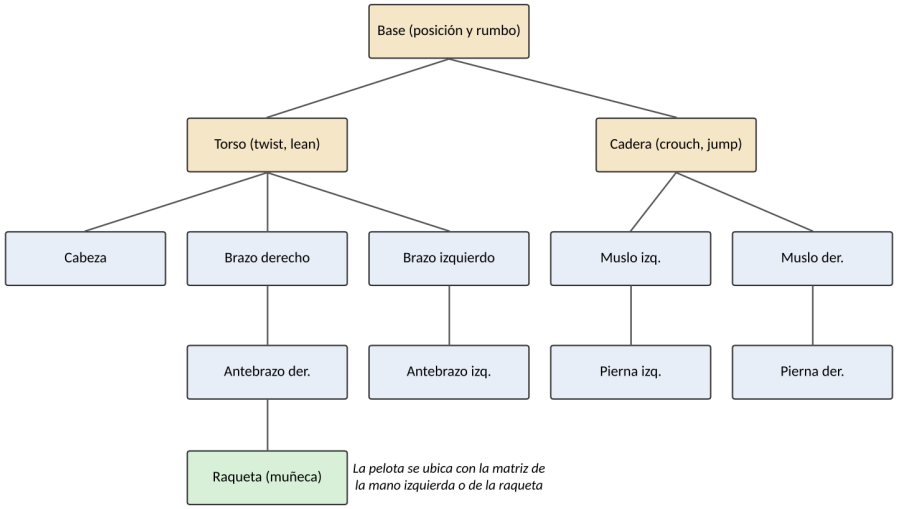


Figura 2. Jerarquía de transformaciones de un jugador.

Diseño de las cámaras

Tecla	Cámara	Descripción
1	Espectador	Cámara libre en primera persona. Comienza en la grada, en la posición 16.3, 4.9, 7.0. Se mueve con WASD y se orienta con el ratón.
2	Orbital	Cámara en tercera persona que gira alrededor del jugador a 7 metros. El ratón cambia el ángulo y la altura.
3	Transmisión TV	Cámara elevada al fondo del estadio, con un campo de visión de 38 grados, que sigue a los jugadores y a la pelota.
4	Jugador	Primera persona desde los ojos del jugador. Se oculta la cabeza para no obstruir la vista.

Tabla 8. Cámaras del proyecto.

Diseño de la iluminación

La iluminación combina tres tipos de luz. La luz direccional representa el sol y su intensidad baja de forma gradual al pasar de día a noche. Cuatro luces puntuales, ubicadas en lo alto de las torres, representan los reflectores del estadio y se encienden con una transición suave. Un reflector central, con cono de 34 grados y borde exterior de 50 grados, ilumina la cancha desde arriba. Al anotar un punto, el público celebra durante unos segundos.

ETAPA 3: IMPLEMENTACIÓN

Herramientas

- GLFW para la ventana, el teclado y el ratón.
- GLEW para cargar las funciones de OpenGL 3.3.
- GLM para vectores, matrices y proyecciones.
- SOIL2 para cargar las texturas.
- Assimp para importar los modelos obj.
- Visual Studio como entorno de desarrollo.

Carga de modelos y texturas

Cada archivo obj se carga con la clase Model, que utiliza Assimp para leer la geometría y el material, y la clase Mesh para guardar los vértices y dibujarlos. Las texturas se cargan con SOIL2 en formato RGBA, con repetición y mipmaps, y se almacenan en una caché compartida para que un mismo archivo no se cargue más de una vez. Cada malla enlaza la textura difusa en la unidad 0 y una textura especular en la unidad 1, que es de un solo píxel cuando el material no la define.

Modelado jerárquico

La función DrawPlayer arma al jugador con matrices anidadas. Cada articulación traslada el origen al punto de giro, aplica la rotación y deja esa matriz como base de las partes hijas. El fragmento siguiente muestra la cadena del brazo derecho, desde el hombro hasta la raqueta.

```
glm::mat4 sh = glm::translate(torso, glm::vec3(-0.23f, 0.52f, 0.0f));
sh = glm::rotate(sh, glm::radians(-p.shRr), glm::vec3(0.0f, 0.0f, 1.0f));
sh = glm::rotate(sh, glm::radians(-p.shRp), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(sh));
M.arm->Draw(shader);
glm::mat4 el = glm::translate(sh, glm::vec3(0.0f, -0.30f, 0.0f));
el = glm::rotate(el, glm::radians(-p.elR), glm::vec3(1.0f, 0.0f, 0.0f));
M.forearm->Draw(shader);
glm::mat4 rk = glm::translate(el, glm::vec3(0.0f, -0.30f, 0.0f));
rk = glm::rotate(rk, glm::radians(p.wrist), glm::vec3(0.0f, 1.0f, 0.0f));
M.racket->Draw(shader);
```

La pose completa del jugador se guarda en una estructura Pose con once valores: inclinación y giro del torso, ángulos de ambos hombros y codos, flexión de piernas, salto y giro de la muñeca. Para que los pies sigan tocando el piso cuando las piernas se flexionan, la altura de la cadera se calcula con la pierna que queda más estirada. La misma cadena de matrices se emplea para ubicar la pelota en la mano izquierda y en el centro de la raqueta, de manera que el objeto siempre acompaña al cuerpo.

Iluminación y día y noche

El fragmento shader reutiliza el modelo de Phong de las prácticas, con una luz direccional, cuatro luces puntuales y un reflector. La variable dayFactor va de uno, que es el día, a cero, que es la noche, y cambia de forma gradual con el paso del tiempo. La intensidad del sol y el color ambiental dependen de ella. La variable reflectoresLevel hace lo mismo con los reflectores, de modo que al pulsar L las luces suben y bajan sin saltos.

```
// Transicion suave dia/noche y de los reflectores
float target = nocheOn ? 0.0f : 1.0f;
if (dayFactor < target) dayFactor = glm::min(target, dayFactor + 0.4f * dt);
else                    dayFactor = glm::max(target, dayFactor - 0.4f * dt);
```

Las lámparas de las torres se dibujan con el shader lamp, al que se le envía el color mediante el uniforme lampColor, de modo que se vean apagadas de día y brillantes cuando los reflectores están encendidos.

Transparencias

La red y las cuerdas de la raqueta usan texturas con canal alfa. Igual que en la práctica de la pelota transparente, se activa la mezcla de colores con GL_SRC_ALPHA y GL_ONE_MINUS_SRC_ALPHA antes de dibujarlos, se envía el uniforme transparency al shader y después se desactiva la mezcla. El shader descarta los fragmentos casi invisibles y conserva el alfa de la textura en la salida.

Cámaras

La cámara del espectador reutiliza la clase Camera de las prácticas. Las cámaras orbital, de televisión y del jugador construyen su matriz de vista con glm::lookAt. En la orbital, la posición se obtiene a partir de dos ángulos y una distancia alrededor del jugador, lo que permite girar con el ratón. En la del jugador, el ojo se ubica a la altura de la cabeza, que se calcula con la altura de la cadera y la pose, y la dirección de la mirada depende del rumbo del jugador y de un ángulo de cabeceo.

```
float py = glm::radians(orbitPitch), yw = glm::radians(orbitYaw);
glm::vec3 target = playerA.pos + glm::vec3(0.0f, 1.2f, 0.0f);
viewPos = target + glm::vec3(orbitDist * cosf(py) * sinf(yw),
                             orbitDist * sinf(py),
                             orbitDist * cosf(py) * cosf(yw));
view = glm::lookAt(viewPos, target, glm::vec3(0.0f, 1.0f, 0.0f));
```

Animaciones básicas y máquina de estados

Varios elementos del escenario se animan con transformaciones sencillas. El público rebota con una función seno cuya fase es distinta para cada persona, y se anima con más energía cuando alguien anota un punto. La cabeza del juez gira para seguir la pelota, con el ángulo calculado a partir de su posición y limitado a 75 grados hacia cada lado. El jugador contrario se desplaza de un lado a otro de la línea de fondo, y los dígitos del marcador cambian de escala al actualizarse.

La caminata de los jugadores usa una máquina de estados como la de la práctica del perro. Una bandera step indica en qué sentido se balancean las piernas. Mientras el jugador camina, la cadera avanza hacia un extremo, al alcanzar el límite cambia la bandera y la cadera regresa hacia el otro, lo que alterna las piernas. Cuando el jugador se detiene, las piernas vuelven a la posición de reposo.

Animación por keyframes: el saque

El saque se construyó con la técnica de la práctica de keyframes. Cada cuadro clave guarda una Pose del jugador, la posición de la pelota y la duración del tramo que le sigue. Se definieron diez cuadros, desde la posición de listo hasta el regreso a la postura de reposo, pasando por el lanzamiento de la pelota, la posición de trofeo, el contacto y el seguimiento.

Cuadro	Pose	Pelota	Duración del tramo
K0	Listo, con la pelota en la mano izquierda.	En la mano	0.70 s
K1	Preparación: las piernas se flexionan y el brazo izquierdo sube.	En la mano	0.50 s
K2	Lanzamiento: se estira el cuerpo y se suelta la pelota.	Se suelta	0.55 s
K3	Trofeo: el brazo de la raqueta llega hacia atrás y arriba.	En el aire	0.32 s
K4	Contacto: el brazo se estira y golpea la pelota.	Centro de la raqueta	0.25 s
K5	Seguimiento: el torso gira y el brazo cruza hacia el otro lado.	En vuelo	0.45 s
K6 a K9	Recuperación de la postura y trayectoria de la pelota hacia el campo contrario.	En vuelo	0.30, 0.45 y 0.60 s

Tabla 9. Cuadros clave del saque.

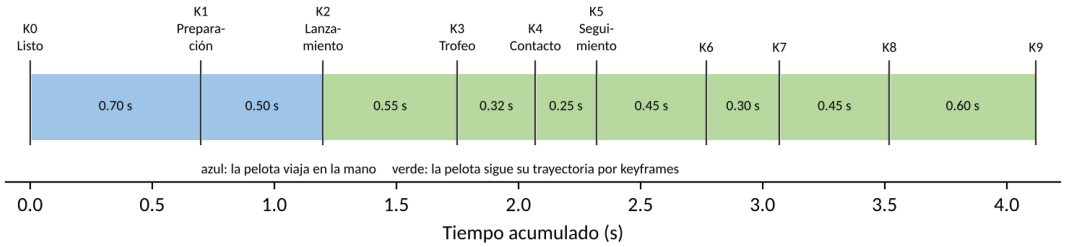


Figura 3. Línea de tiempo del saque. La duración total es de 4.12 segundos.

Durante la reproducción se acumula el tiempo del tramo actual. Cuando supera la duración del cuadro se pasa al siguiente, y con el tiempo restante se calcula el factor de interpolación, que se aplica a cada valor de la pose y a la posición de la pelota. Mientras la pelota no se ha soltado, se coloca en la mano con la cinemática de la pose, y después sigue su propia trayectoria.

```
Pose LerpPose(const Pose& a, const Pose& b, float s)
{
    Pose r;
    float* ra = (float*)&r;
    const float* pa = (const float*)&a;
    const float* pb = (const float*)&b;
    for (int i = 0; i < 11; i++)
        ra[i] = pa[i] + (pb[i] - pa[i]) * s;
    return r;
}
...
float s = tSeg / KeyFrame[playIndex].dur;
playerA.pose = LerpPose(a.pose, b.pose, s);
if (playIndex < RELEASE_FRAME)
    playerA.ball = LeftHand(playerA.pose);
else
    playerA.ball = a.ball + (b.ball - a.ball) * s;
```

Como la interpolación depende del tiempo transcurrido y no del número de cuadros, el saque dura lo mismo en cualquier computadora.

Interacción y marcador

El teclado concentra las interacciones. L enciende y apaga los reflectores, N alterna entre día y noche, la barra espaciadora inicia el saque, y P y O suman un punto al jugador uno o al jugador dos. El marcador es un tablero con dígitos dibujados como cuadros texturizados, que muestran los puntos en el game actual, con las formas 0, 15, 30, 40 y ventaja, y los games ganados por cada jugador. Al sumar un punto el público celebra, y R reinicia el marcador. Además, el título de la ventana funciona como un panel de información con la cámara activa, el marcador y el estado de la luz.

Navegación libre

Con la cámara del espectador el usuario recorre libremente el estadio. Con las cámaras orbital y del jugador, las teclas WASD mueven al jugador sobre su lado de la cancha, con límites que le impiden atravesar la red o salir del área de juego, y la caminata se anima de forma automática.

Modo de demostración

Para grabar el video se agregó un recorrido automático. Se activa con la tecla V o al iniciar el programa con el argumento --demo, y ejecuta una serie de eventos en el tiempo: movimiento de la cámara libre por las tres zonas, cambio entre las cuatro cámaras, un saque de día, reflectores, noche, puntos y un segundo saque desde la cámara del jugador. Dura aproximadamente un minuto.

ETAPA 4: PRUEBAS

Las pruebas se realizaron ejecutando el programa y comprobando de forma visual el resultado de cada función. En el título de la ventana se verificó el estado de la cámara, la luz y el marcador.

Prueba	Procedimiento	Resultado esperado	Resultado
Carga de modelos	Iniciar el programa.	Se cargan los 52 modelos y sus texturas sin errores en consola.	Correcto
Cambio de cámara	Pulsar 1, 2, 3 y 4.	La vista cambia a cada cámara y el título lo indica.	Correcto
Navegación libre	Cámara 1 con WASD y ratón.	La cámara recorre el estadio y mira hacia donde apunta el ratón.	Correcto
Movimiento del jugador	Cámara 2 o 4 con WASD.	El jugador camina con las piernas alternadas y no cruza la red.	Correcto
Saque	Pulsar la barra espaciadora.	La pelota sale de la mano, pasa por la posición de trofeo, se golpea con la raqueta y vuela hacia la otra cancha.	Correcto
Día y noche	Pulsar N.	La luz solar baja y sube de forma gradual.	Correcto
Reflectores	Pulsar L de noche.	Se iluminan la cancha y las gradas, y las lámparas se ven encendidas.	Correcto
Marcador	Pulsar P y O varias veces.	Los dígitos pasan por 15, 30, 40 y el game, y el público celebra.	Correcto
Transparencia	Mirar la red y la raqueta.	Se ven las cuerdas sin zonas negras.	Correcto
Juez de silla	Hacer un saque.	La cabeza del juez sigue a la pelota.	Correcto
Recorrido automático	Ejecutar con --demo.	El recorrido completo corre sin intervención y termina en un minuto.	Correcto

Tabla 10. Casos de prueba funcionales.

Problemas encontrados y soluciones

Problema	Solución
La red se veía como una banda negra, porque el mipmap del canal alfa superaba el umbral de descarte.	Activar la mezcla de colores al dibujar la red y la raqueta, como en la práctica de la pelota transparente.
De noche la cancha se veía demasiado saturada.	Reducir la intensidad difusa, ambiental y especular de las luces puntuales y del reflector.
La cámara inicial quedaba dentro de la cabeza de un espectador.	Dejar pasillos sin espectadores en las gradas y mover la cámara inicial.
En primera persona se veía la propia cabeza del jugador durante el seguimiento.	Agregar un parámetro a DrawPlayer para omitir la cabeza con la cámara del jugador.
El recorrido libre pasaba por detrás del marcador y se veía su parte trasera sin textura.	Definir nuevos puntos de paso para la cámara del recorrido automático.

Tabla 11. Problemas encontrados durante las pruebas y su solución.

RESULTADOS

Las siguientes capturas provienen del video demostrativo y muestran las tres zonas del escenario, las cuatro cámaras y las condiciones de día y de noche.

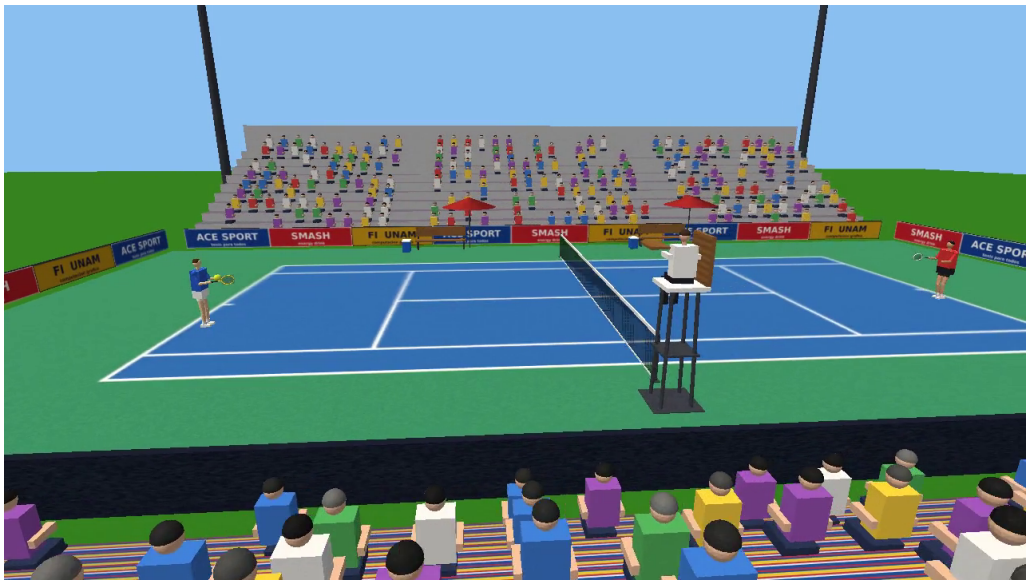


Figura 4. Vista general del estadio desde la cámara del espectador.

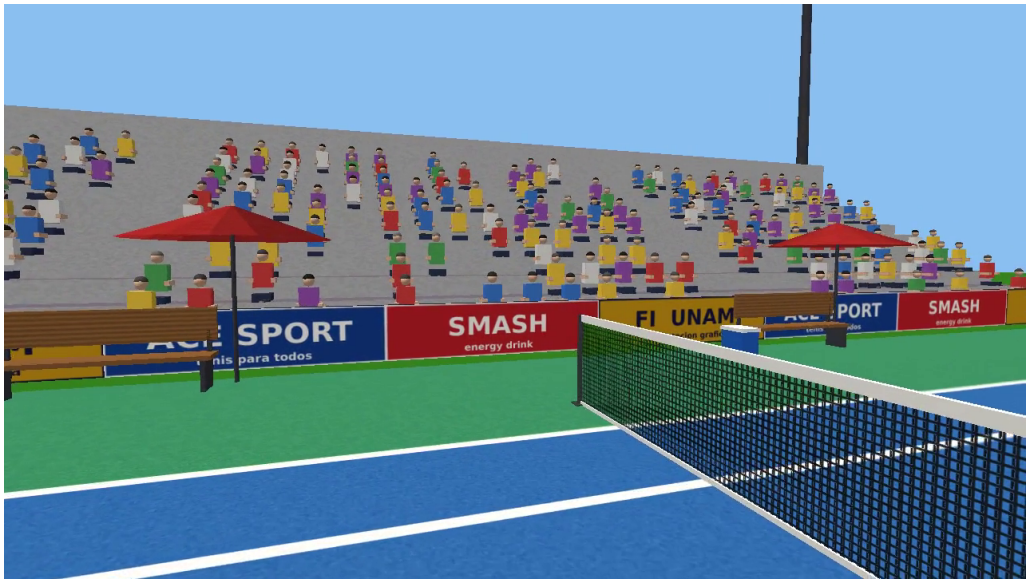


Figura 5. Zona 2: bancas de los jugadores con hielera y sombrilla.



Figura 6. Cámara de transmisión de televisión durante el saque, de día.



Figura 7. Escenario de noche con los reflectores encendidos.



Figura 8. Cámara orbital durante el saque.



Figura 9. Posición de trofeo y contacto del saque, con el modelado jerárquico del jugador.



Figura 10. Cámara en primera persona del jugador.



Figura 11. Cámara de televisión con el marcador actualizado.

MANUAL DE USO

Tecla o acción	Función
1, 2, 3, 4	Seleccionar la cámara: espectador, orbital, televisión y jugador.
W A S D o flechas	Mover la cámara del espectador o al jugador. En la cámara orbital, las flechas izquierda y derecha giran la vista.
Ratón	Mirar alrededor u orbitar al jugador.
Barra espaciadora	Ejecutar el saque.
L	Encender o apagar los reflectores.
N	Cambiar entre día y noche.
P y O	Sumar un punto al jugador uno o al jugador dos.
R	Reiniciar el marcador.
V	Iniciar o detener el recorrido automático.
Esc	Salir del programa.

Tabla 12. Controles del programa.

Compilación

- Crear un proyecto de consola en Visual Studio y agregar main.cpp, Model.h, Mesh.h, Shader.h y Camera.h.
- Vincular GLFW, GLEW, SOIL2 y Assimp de la misma forma que en las prácticas, y agregar opengl32.lib.

- Colocar las carpetas Models y Shader junto al ejecutable, o configurar el directorio de trabajo en la raíz del proyecto.
- Compilar y ejecutar. Para el recorrido automático, usar el argumento de línea de comandos --demo.

CÓDIGO FUENTE Y VIDEO DEMOSTRATIVO

El código fuente completo del proyecto, con los modelos, las texturas, los shaders y el generador de modelos, está disponible en la siguiente liga de descarga. El video muestra el funcionamiento del proyecto, incluyendo las zonas, las cámaras, la iluminación y las animaciones.

Código fuente	PEGAR AQUÍ LA LIGA DE DESCARGA DEL CÓDIGO FUENTE
Video demostrativo	PEGAR AQUÍ LA LIGA DEL VIDEO REPRODUCIBLE

Estructura del paquete de código

```
Proyecto_Tenis3D/
  main.cpp               programa principal
  Model.h  Mesh.h        carga de modelos con Assimp y SOIL2
  Shader.h  Camera.h     clases de las practicas
  Shader/                lighting.vs, lighting.frag, lamp.vs, lamp.frag
  Models/                52 modelos obj con sus mtl y 42 texturas png
  tools/                 generar_modelos.py
  CMakeLists.txt         compilacion alternativa en Linux
  README.md              instrucciones
```

CONCLUSIONES

El proyecto permitió integrar en un solo programa las técnicas vistas durante el curso. El modelado jerárquico mostró su utilidad al animar a los jugadores, porque una misma cadena de matrices sirve para dibujar el cuerpo y para ubicar la pelota en la mano y en la raqueta. La iluminación con tres tipos de luz y la transición entre día y noche cambian por completo el aspecto del estadio sin modificar la geometría.

La animación por keyframes resultó adecuada para movimientos con intención, como el saque, ya que cada pose se ajusta a mano y la interpolación se encarga del resto. En cambio, la máquina de estados fue la herramienta más simple para la caminata, que es un movimiento cíclico. Combinar ambas técnicas en el mismo jugador fue una de las partes más instructivas del trabajo.

Scrum ayudó a mantener el trabajo ordenado. Al terminar cada fase se contaba con un programa que se podía ejecutar, y eso facilitó detectar problemas de integración, como la transparencia de la red o la posición inicial de la cámara, mientras todavía quedaba tiempo para corregirlos. Como mejora futura se podrían agregar sombras, física para la pelota, sonido y las reglas completas del tenis, que quedaron fuera del alcance definido en el acta.

REFERENCIAS

Anderson, D. J. (2010). *Kanban: Successful evolutionary change for your technology business*. Blue Hole Press.

Beck, K., y Andres, C. (2004). *Extreme programming explained: Embrace change* (2.ª ed.). Addison-Wesley.

Fuerte Martínez, E. (2026). *Proyecto Final 2027-1: Escenario deportivo 3D interactivo* [Documento del curso]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, *26*(8), 1-9.

Schwaber, K., y Sutherland, J. (2020). *La guía de Scrum*. https://scrumguides.org/scrum-guide.html

de Vries, J. (2020). *Learn OpenGL*. https://learnopengl.com/

Equipo del proyecto. (2026). *Acta del proyecto: Escenario deportivo 3D interactivo de tenis* [Documento del curso]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

The Khronos Group. (2010). *The OpenGL graphics system: A specification*, *version 3.3*. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf